T7 - PyTorch Basics: a tiny neural network, trained and plotted

Covers: tensors, `nn.Module`, `nn.Linear`, ReLU/Sigmoid, loss + optimiser, the 5-line training loop, loss recording and plotting, evaluation, DataLoader basics. Earns the 12 marks of 2024 "Python Task 3" (3a 4 + 3b 2 + 3c 3 + 3d 3). The mock exam may replace this task by a CSV/pandas/numpy/matplotlib task - see section 8.

1. What the exam asks

Only the **2024 paper** (and the course labs) use PyTorch. The 2021, 2022 and 2023 Python parts were tkinter + database tasks (see T8). The "Practice Exam Task 3" document is titled "*Practice Task 3 (replacing pytorch)*", which tells you the examiners may swap PyTorch for CSV/pandas/numpy/matplotlib.

2024 Final Lab Exam, Python Task 3: Basic Neural Network Implementation [12 marks]

> Define a class `SimpleNN` that extends `torch.nn.Module`. Input Layer: 10 input features. Hidden Layer 1: fully connected, 5 neurons, ReLU. Hidden Layer 2: fully connected, 3 neurons, ReLU. Output Layer: fully connected, 1 neuron, Sigmoid.

Part	Marks	Wording
3a	4	Instantiate `SimpleNN`. Use `torch.nn.MSELoss()` as loss. Use `torch.optim.SGD` with learning rate 0.01.
3b	2	Synthetic data: 100 samples, 10 features. Target is "a tensor with 100 values". Use `torch.randn` for input **and** target.
3c	3	Train 20 epochs. Each epoch: compute loss, backpropagate, update weights. **Record the training loss for each epoch.**
3d	3	Plot the training loss over epochs using matplotlib. "*Your plot should display the training loss decreasing over time, indicating that the model is learning.*"

Model answer in the course folder: `Practice/LabExam2024-25/ProgSD-Python-Solution.py` (Task 3a-3d). Course lab code: `WEEK2/tAPP_Lab03/Lab3_sol/pytorch/ex_2.py` - `ex_4.py` (MNIST) and `CIFAR.py`; notes in `pytorch.py`; extra practice in `PyTorch_neural_network_Practical exercises.docx` (RevenueNet regression with MSELoss + Adam; RiskNN binary classification with BCELoss + SGD).

2. Mental model (analogy)

Think of the network as **a student revising for quizzes**.

PyTorch thing	Analogy
Tensor	A grid of numbers (a spreadsheet). Shape `(100, 10)` = 100 rows (samples), 10 columns (features).
Model weights	The student's *study habits* (77 of them in `SimpleNN`). Start random.
`model(data)` forward pass	The student *sits the quiz* and writes answers.
`loss = criterion(output, target)`	The quiz **score**, counted as *points lost*. Lower = better. One number.
`loss.backward()`	The teacher **marks the paper** and works out *which habits cost points, and by how much* (the gradients).
`optimizer.step()`	The student **adjusts their study habits** using the teacher's feedback.
`optimizer.zero_grad()`	**Wiping the whiteboard**: rub out last round's feedback, otherwise the new feedback is added on top of the old (PyTorch *accumulates* gradients).
Learning rate `lr`	How *big* an adjustment the student makes. Too small: crawls. Too big: overreacts, scores jump around.
Epoch	One full round of the quiz (here: all 100 samples once).
SGD / Adam	SGD: same-size nudge to every habit. Adam: a student with a memory who adapts the nudge per habit (usually learns faster).
ReLU	A one-way gate: lets positive numbers through, turns negatives into 0.
Sigmoid	A dimmer dial: squashes any number into the range 0 to 1.
`torch.no_grad()` + `model.eval()`	The **real exam hall**: just answer, no feedback, no changing habits.
Train / test split	Practice questions (learn from them) vs the unseen exam questions (only to measure).
DataLoader	A waiter bringing the data in **trays** of `batch_size` rows instead of the whole buffet.

Reading a layer spec. "10 inputs, 5, 3, 1" is a list of *sizes*. Each `nn.Linear(in, out)` joins **two neighbouring numbers**; the `out` of one layer is the `in` of the next.

```
spec:      10  ->  5  ->  3  ->  1
layers:       Linear(10,5)   Linear(5,3)   Linear(3,1)
              fc1            fc2           fc3
```

Rule: **n numbers in the spec = n-1** `nn.Linear` **layers.** The first number is the number of input columns in your data.

3. Copy-paste templates

3.1 The exam answer: `SimpleNN` 10-5-3-1 (use for Task 3a-3d)

```python
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

torch.manual_seed(42)   # optional: same random numbers on every run


class SimpleNN(nn.Module):
    def __init__(self):
        super(SimpleNN, self).__init__()
        self.fc1 = nn.Linear(10, 5)   # input 10 -> hidden 1 (5)
        self.fc2 = nn.Linear(5, 3)    # hidden 1 (5) -> hidden 2 (3)
        self.fc3 = nn.Linear(3, 1)    # hidden 2 (3) -> output (1)

    def forward(self, x):
        x = torch.relu(self.fc1(x))      # hidden 1 + ReLU
        x = torch.relu(self.fc2(x))      # hidden 2 + ReLU
        x = torch.sigmoid(self.fc3(x))   # output + Sigmoid
        return x


# Task 3a: model, loss function, optimiser
model = SimpleNN()
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

# Task 3b: synthetic data (target shape (100, 1) matches the output)
data = torch.randn(100, 10)     # 100 samples, 10 features
target = torch.randn(100, 1)    # 100 target values

# Task 3c: training loop, recording the loss of every epoch
losses = []
for epoch in range(20):
    optimizer.zero_grad()                # 1. wipe old gradients
    output = model(data)                 # 2. forward pass
    loss = criterion(output, target)     # 3. compute the loss
    loss.backward()                      # 4. backpropagation
    optimizer.step()                     # 5. update the weights
    losses.append(loss.item())           # record as a plain float
    print(f"Epoch {epoch + 1:02d} | Loss: {loss.item():.6f}")

# Task 3d: plot the loss curve
plt.plot(range(1, 21), losses, marker="o")
plt.title("Training Loss Over Time")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.show()
```

Real output with seed 42 (first line, last line): `Epoch 01 | Loss: 1.195504` ... `Epoch 20 | Loss: 1.191171`. It goes *down*, but only by 0.004 - see section 4b for why, and what to do.

3.2 Check the model before training (30 seconds, catches shape bugs)

```python
import torch
import torch.nn as nn

model = nn.Sequential(          # short alternative to a class
    nn.Linear(10, 5), nn.ReLU(),
    nn.Linear(5, 3), nn.ReLU(),
    nn.Linear(3, 1), nn.Sigmoid(),
)
print(model)                                # lists every layer
```

```
print(model(torch.randn(4, 10)).shape)       # torch.Size([4, 1])
print(sum(p.numel() for p in model.parameters()))   # 77 weights+biases
```

TIP: the exam says "define a class that extends `nn.Module`", so *submit the class version* (3.1). `nn.Sequential` is only a quick way to test a spec. Output shape is `(batch, last_number)`: 4 rows in -> `(4, 1)` out.

3.3 Loss + optimiser + target pairing (decide in 10 seconds)

Task	Last layer	Loss	Target
Predict a number (regression)	none	`nn.MSELoss()`	float, **same shape as output**, e.g. `(N, 1)`
Yes/no (binary)	`torch.sigmoid`	`nn.BCELoss()`	float 0./1., **same shape**, `(N, 1)`
Pick 1 of C classes	**none** (raw scores)	`nn.CrossEntropyLoss()`	**long** class numbers, shape `(N,)`

Optimiser line: `optim.SGD(model.parameters(), lr=0.01)` or `optim.Adam(model.parameters(), lr=0.01)`.

3.4 A target the network CAN learn (loss visibly drops)

Use this as an optional *second run* or when asked to show learning (reasons in 4b). Run with the `SimpleNN` class from 3.1 defined.

```
# --- needs SimpleNN from 3.1 above ---
torch.manual_seed(42)
model2 = SimpleNN()
criterion = nn.MSELoss()
optimizer = optim.Adam(model2.parameters(), lr=0.05)   # Adam, bigger lr

data = torch.randn(100, 10)
# target IS a function of the input: sigmoid of a sum of the features
target = torch.sigmoid(data.sum(dim=1, keepdim=True))  # shape (100, 1)

losses2 = []
for epoch in range(20):
    optimizer.zero_grad()
    loss = criterion(model2(data), target)
    loss.backward()
    optimizer.step()
    losses2.append(loss.item())
print(round(losses2[0], 3), "->", round(losses2[-1], 3))
```

For regression practice (RevenueNet, `PyTorch_neural_network_Practical exercises.docx`) the learnable target is a noisy linear mix; for binary (RiskNN) it is a thresholded sigmoid:

```
import torch

X = torch.randn(120, 8)
w = torch.randn(8, 1)
y_reg = X @ w + 0.5 + 0.1 * torch.randn(120, 1)        # regression target
proba = torch.sigmoid(X @ w + 0.3 * torch.randn(120, 1))
y_bin = (proba > 0.5).float()                          # 0./1. labels (120, 1)
print(X.shape, y_reg.shape, y_bin.shape)
```

3.5 Mini-batches with a DataLoader (as in `CIFAR.py` / `ex_3.py`)

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader

data = torch.randn(100, 10)
target = torch.sigmoid(data.sum(dim=1, keepdim=True))
dataset = TensorDataset(data, target)             # pairs row i of each
loader = DataLoader(dataset, batch_size=10, shuffle=True)   # 10 trays

model = nn.Sequential(nn.Linear(10, 5), nn.ReLU(),
                      nn.Linear(5, 3), nn.ReLU(),
                      nn.Linear(3, 1), nn.Sigmoid())
criterion = nn.MSELoss()
```

```
optimizer = optim.SGD(model.parameters(), lr=0.01)

losses = []
for epoch in range(20):
    running_loss = 0.0
    for inputs, targets in loader:          # one tray at a time
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    losses.append(running_loss / len(loader))   # mean loss of the epoch
print(len(loader), "batches per epoch;", round(losses[0], 3), "->",
      round(losses[-1], 3))
```

`len(loader)` = number of batches = 100 / 10 = 10. Each epoch now makes **10 weight updates instead of 1**, so it learns faster.

3.6 Train / test split and evaluation (as in `ex_4.py`)

```
import torch
import torch.nn as nn
import torch.optim as optim

torch.manual_seed(0)
data = torch.randn(100, 10)
labels = (data.sum(dim=1, keepdim=True) > 0).float()    # 0./1., (100, 1)
train_x, test_x = data[:80], data[80:]                  # 80 / 20 split
train_y, test_y = labels[:80], labels[80:]

model = nn.Sequential(nn.Linear(10, 5), nn.ReLU(),
                      nn.Linear(5, 1), nn.Sigmoid())
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.05)

model.train()                           # practice mode
for epoch in range(100):
    optimizer.zero_grad()
    loss = criterion(model(train_x), train_y)
    loss.backward()
    optimizer.step()

model.eval()                            # exam-hall mode
with torch.no_grad():                   # no gradients, no learning
    outputs = model(test_x)
    predicted = (outputs > 0.5).float()                 # 0./1. guesses
    correct = (predicted == test_y).sum().item()        # how many right
    test_loss = criterion(outputs, test_y).item()
print(f"Test accuracy: {100 * correct / len(test_y):.1f}%")
print(f"Test loss: {test_loss:.4f}")
```

For several classes (`CrossEntropyLoss`) the prediction is the index of the biggest score, exactly as in `ex_4.py`:

```
import torch
import torch.nn as nn

model = nn.Linear(4, 3)                         # 4 features -> 3 classes
x = torch.randn(6, 4)
labels = torch.tensor([0, 1, 2, 1, 0, 2])       # long, shape (6,)
outputs = model(x)                              # raw scores, shape (6, 3)
loss = nn.CrossEntropyLoss()(outputs, labels)   # no softmax before this!
_, predicted = torch.max(outputs.data, 1)       # index of the best score
correct = (predicted == labels).sum().item()
print(outputs.shape, round(loss.item(), 3), correct, "of", len(labels))
```

3.7 Plot template (matplotlib, as in `pytorch.py`)

```
import matplotlib.pyplot as plt
```

```
losses = [1.2, 1.1, 1.05, 1.0, 0.98]       # your list from the loop
plt.plot(range(1, len(losses) + 1), losses, marker="o")
plt.title("Training Loss Over Time")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.show()
```

4. Line-by-line walkthrough (the exam answer from 3.1)

```
import torch                        # tensors + autograd (the maths engine)
import torch.nn as nn               # nn.Module, nn.Linear, nn.MSELoss
import torch.optim as optim         # optimisers: SGD, Adam
import matplotlib.pyplot as plt     # for the loss curve

torch.manual_seed(42)               # fix the dice: same random numbers


class SimpleNN(nn.Module):          # our network IS-A nn.Module (inherits it)
    def __init__(self):             # runs once: builds the layers
        super(SimpleNN, self).__init__()   # MUST be first: sets up nn.Module
        self.fc1 = nn.Linear(10, 5)   # layer 1: 10 numbers in, 5 out
        self.fc2 = nn.Linear(5, 3)    # layer 2: takes the 5, gives 3
        self.fc3 = nn.Linear(3, 1)    # layer 3: takes the 3, gives 1

    def forward(self, x):           # how data flows; called by model(x)
        x = torch.relu(self.fc1(x))      # fc1 then ReLU gate: negatives -> 0
        x = torch.relu(self.fc2(x))      # same again for hidden layer 2
        x = torch.sigmoid(self.fc3(x))   # squash the result into 0..1
        return x                         # one number per sample
```

- `fc` = "fully connected" = `nn.Linear`. The layers hold the **study habits** (weights); `forward` is the **route the answer takes** through them.
- `super(SimpleNN, self).__init__()` is the course style (`ex_2.py`); `super().__init__()` is identical. Forget it and you get `AttributeError: cannot assign module before Module.__init__() call`.
- Name it `forward` exactly. You never call it directly: `model(x)` calls it for you.

```
model = SimpleNN()                  # student created with random habits
criterion = nn.MSELoss()            # marking scheme: mean of (guess-answer)^2
optimizer = optim.SGD(model.parameters(), lr=0.01)  # who adjusts, how big
```

- `model.parameters()` hands the optimiser **all the habits it may change**.
- MSE = *mean squared error*: square every miss, then average. One number out.

```
data = torch.randn(100, 10)         # 100 quiz papers, 10 questions each
target = torch.randn(100, 1)        # the 100 "right answers", one column
```

- `torch.randn` = random numbers from a bell curve (mean 0, spread 1).
- Target is `(100, 1)`, **not** `(100,)`, so it matches the model output `(100, 1)` (trap 5.3).

```
losses = []                         # the score-book: one entry per epoch
for epoch in range(20):             # 20 rounds of the quiz
    optimizer.zero_grad()           # WIPE the whiteboard (old feedback)
    output = model(data)            # student sits the quiz: shape (100, 1)
    loss = criterion(output, target)   # score = points lost, ONE number
    loss.backward()                 # teacher marks: which habit cost what
    optimizer.step()                # student adjusts habits by lr * feedback
    losses.append(loss.item())      # .item(): tensor -> plain Python float
    print(f"Epoch {epoch + 1:02d} | Loss: {loss.item():.6f}")
```

- The five numbered lines are **always in this order**. Memorise: *zero, forward, loss, backward, step*.
- The loss is recorded for the quiz *just sat*, before the adjustment (so epoch 1 shows the untrained score).
- `.item()` turns a one-number tensor into a float. A list of floats plots with no fuss; a list of tensors may fail.
- `{epoch + 1:02d}` prints 01..20 (epochs counted from 1); `{:.6f}` = 6 decimals.

```
plt.plot(range(1, 21), losses, marker="o")   # x = epoch 1..20, y = loss
plt.title("Training Loss Over Time")         # the labels earn the marks
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.show()                                   # open the window
```

4b. Why the loss barely moves with random targets (and what to do)

Run the spec exactly and the loss falls only a hair, e.g. `1.1955 –> 1.1912` (seed 42). That is **expected, not a bug.** Three reasons:

1. **Only 20 updates.** The loop uses the whole dataset at once (full batch), so one epoch = **one** weight update, with lr 0.01. 20 tiny steps.
2. **Nothing to learn.** `torch.randn` targets are random noise with no link to the inputs. The best the student can do is guess the average; the loss can never fall far below the noise level (about 1 for randn targets).
3. **Sigmoid can only say 0..1** but about half the random targets are negative, so the floor is even higher.

What I measured (SGD lr 0.01, 20 epochs, spec data): the loss went down in **200 out of 200 random seeds**, but by only 0.0015-0.04. Matplotlib **auto-zooms the y-axis**, so the curve *still looks like a clean downward line* (y-axis labels such as 1.1920 ... 1.1950). The model answer plots exactly this.

What to do in the exam:

1. Submit the spec exactly: SGD, lr 0.01, MSELoss, 20 epochs, `randn` data. That is where the marks are (3a: 4 marks).
2. Add one comment line above the plot, e.g. `# loss falls slowly: random targets + 20 full-batch SGD steps (lr=0.01)`.
3. If you want a *dramatic* decrease as a bonus, add the second run from **3.4** (target = `sigmoid(sum of features)` + Adam lr 0.05: loss 0.176 -> 0.039) or 3.5 (mini-batches: 10 updates per epoch). My measurements:

Setup (20 epochs, seed 42)	Loss start -> end
Spec: random target, SGD 0.01, full batch	1.1955 -> 1.1912 (flat)
Random target, SGD 0.01, batch size 10	1.1948 -> 1.1654 (a bit clearer)
Random target, Adam 0.05, full batch	1.1955 -> 1.0000 (clear)
Learnable target, SGD 0.01, full batch	0.1757 -> 0.1724 (flat)
Learnable target, SGD 0.01, batch size 10	0.1750 -> 0.1506
Learnable target, Adam 0.05, full batch	0.1757 -> 0.0389 (clear)
Learnable target, Adam 0.05, batch size 10	0.1313 -> 0.0027 (dramatic)

Numbers differ with the seed; the pattern does not. Fixing the seed with `torch.manual_seed(42)` makes your numbers repeatable, which helps when comparing runs.

5. Traps & debugging

5.1 Forgot `super().__init__()`

```
AttributeError: cannot assign module before Module.__init__() call
```

Fix: first line inside `__init__`: `super(SimpleNN, self).__init__()`.

5.2 Wrong input size (data columns not equal to first `Linear`)

```
RuntimeError: mat1 and mat2 shapes cannot be multiplied (100x9 and 10x5)
```

Read it as *(your data) and (layer weights)*. Data has 9 columns, `fc1` expects 10. Fix the data or the first `nn.Linear(10, ...)`.

5.3 Target shape `(100,)` vs output `(100, 1)` (silent bug!) `torch.randn(100)` is a *flat* list of 100. The model output is `(100, 1)` (a column). `MSELoss` does **not crash**: it prints a `UserWarning: Using a target size (torch.Size([100])) that is different to the input size ...` and *broadcasts* both to a 100 x 100 grid, so it compares every output with every target. The loss number is **wrong** (I got 0.7450 instead of 0.7471 for the same model). Fixes, any one of:

```
target = torch.randn(100, 1)            # best: create it as a column
loss = criterion(output.squeeze(1), t)  # or flatten the output
loss = criterion(output, t.view(-1, 1)) # or reshape the target
```

Exam wording "a tensor with 100 values" = 100 numbers; a `(100, 1)` column holds 100 values and matches the output.

5.4 Forgot `optimizer.zero_grad()` No error. Gradients pile up from epoch to epoch, so updates grow too large and the loss may bounce or rise. Fix: first line of the loop.

5.5 Appending the tensor, not the number `losses.append(loss)` keeps a tensor tied to the graph (memory grows, and `plt.plot` may raise `Can't call numpy() on Tensor that requires grad`). Fix: `losses.append(loss.item())`.

5.6 Calling `.item()` on many values

```
RuntimeError: a Tensor with 3 elements cannot be converted to Scalar
```

`.item()` only works on a one-element tensor (a loss is; an output of 100 rows is not).

5.7 BCELoss problems

```
RuntimeError: Found dtype Long but expected Float
RuntimeError: all elements of input should be between 0 and 1
```

Targets must be floats (`.float()`), and the model output must be passed through `sigmoid` first.

5.8 CrossEntropyLoss problems

```
RuntimeError: expected target dtype to be Long or Byte, but got Float
IndexError: Target 3 is out of bounds.
```

Labels must be integer class numbers `0..C-1` (no `.float()`), and the last layer must output C scores. **NOTE:** `CrossEntropyLoss` applies softmax itself, so give it raw scores. (The lab files `ex_2.py` - `ex_4.py` end the model with `torch.softmax` and then use `CrossEntropyLoss`, i.e. softmax twice. It still trains, but for new code leave softmax out.)

5.9 Evaluating the wrong way Forgetting `torch.no_grad()` is not an error, only slow and wasteful. Doing `loss.backward()` / `optimizer.step()` on the test set is a *logic* error: you would be learning from the exam paper.

5.10 Plot is empty / nothing appears Put `plt.show()` *after* the loop, and plot a list (`losses`), not one number.

6. Anticipated exam questions

Q1 [4] Define `SimpleNN` with layers 10 -> 5 (ReLU) -> 3 (ReLU) -> 1 (Sigmoid). Answer: class in 3.1: three `nn.Linear` in `__init__` (after `super().__init__()`), and `torch.relu` / `torch.relu` / `torch.sigmoid` in `forward`, returning `x`.

Q2 [2] Create the model, `MSELoss` and SGD with lr 0.01.

```
model = SimpleNN()
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)
```

Q3 [2] Generate 100 samples with 10 features and 100 target values using `torch.randn`.

```
data = torch.randn(100, 10)
target = torch.randn(100, 1)
```

Q4 [3] Train for 20 epochs and record each epoch's loss. Loop in 3.1: `zero_grad, forward, criterion, backward, step`, then `losses.append(loss.item())`.

Q5 [3] Plot the loss against epoch with labels and title. 3.7: `plt.plot(...)`, `title`, `xlabel`, `ylabel`, `plt.show()`.

Q6 [2] Your loss only falls from 1.1955 to 1.1912. Is the model learning? Why so little? Yes, it is moving in the right direction. Targets are random noise (nothing to learn), only 20 full-batch updates at lr 0.01 are made, and the sigmoid output cannot reach negative targets. A learnable target, Adam or mini-batches show a larger drop (3.4, 3.5).

Q7 [2] What does `optimizer.zero_grad()` do and why is it needed? It resets the stored gradients to zero. PyTorch *adds* new gradients to old ones on every `backward()`, so without it each update would use stale, accumulated feedback.

Q8 [2] Why `loss.item()`? It converts the one-value loss tensor to a normal Python float, so the list can be plotted and does not keep the autograd graph alive.

Q9 [3] Evaluate on unseen data. Split data (e.g. 80/20), train on the first part, then `model.eval()` and `with torch.no_grad():` compute outputs/accuracy on the second part (3.6).

Q10 [2] Which loss for (a) predicting house price, (b) spam yes/no, (c) 10 digit classes? (a) `MSELoss`; (b) `BCELoss` with sigmoid output; (c) `CrossEntropyLoss` with 10 raw outputs and integer labels.

7. Quick-reference box (copy onto the notes sheet)

```
import torch, torch.nn as nn, torch.optim as optim
import matplotlib.pyplot as plt
class Net(nn.Module):
  def __init__(self):
    super(Net, self).__init__()       # FIRST LINE, always
    self.fc1 = nn.Linear(10, 5)       # spec "10,5,3,1": pairs of numbers
    self.fc2 = nn.Linear(5, 3); self.fc3 = nn.Linear(3, 1)
  def forward(self, x):               # name must be forward; return x
    x = torch.relu(self.fc1(x)); x = torch.relu(self.fc2(x))
    return torch.sigmoid(self.fc3(x))
model = Net(); criterion = nn.MSELoss()   # BCELoss / CrossEntropyLoss
optimizer = optim.SGD(model.parameters(), lr=0.01)  # or optim.Adam
data = torch.randn(100, 10); target = torch.randn(100, 1)  # (100,1) !
losses = []
for epoch in range(20):
  optimizer.zero_grad(); out = model(data)   # wipe -> forward
  loss = criterion(out, target)              # score
  loss.backward(); optimizer.step()          # mark -> adjust
  losses.append(loss.item())                 # .item() = float
plt.plot(losses); plt.title("..."); plt.xlabel("Epoch")
plt.ylabel("Loss"); plt.show()
EVAL: model.eval(); with torch.no_grad(): out = model(test_x)
 binary: (out > 0.5).float() | multi: _, pred = torch.max(out, 1)
```

```
BATCHES: DataLoader(TensorDataset(X, y), batch_size=10, shuffle=True)
 for xb, yb in loader: (same 5 lines)   len(loader) = no. of batches
PAIRS: MSE -> float same shape | BCE -> sigmoid + float | CE -> raw + long
Flat loss with randn target = EXPECTED. torch.manual_seed(42) for repeats.
```

8. If PyTorch is replaced

The *Practice Exam Task 3* document is headed "replacing pytorch": a CSV file read and written with pandas/numpy, summary statistics saved to a text file, and a matplotlib histogram. If the mock exam does this, nothing in this document is needed except the **plotting habit** (3.7: title, both axis labels, `plt.show()`) and the general loop/list/function idioms. Go instead to **T3, T5 and T6** (the pandas/CSV, numpy and matplotlib topic documents in this folder; check the pack index for exact titles) and the solution documents for the full worked tasks. A sensible split of last-minute revision time: T3/T5/T6 first, SQLite second, this document third, T8 (tkinter) never.